Git Interview Questions — Answers

Model answers for the 20 questions, organized by section

Basics

1. What is Git, and how does it differ from a centralized version control system like SVN?

Git is a distributed version control system: every clone contains the full history of the repository, not just a working copy. This means most operations (commits, diffs, log, branching) happen locally and don't need network access. In centralized systems like SVN, there's a single central repository, and clients hold only a working copy of one revision — history and most operations require contacting the server. Git's distributed model also makes branching cheap and enables fully offline workflows.

2. What is the difference between 'git fetch' and 'git pull'?

'git fetch' downloads commits, files, and refs from a remote into your local repository's remote-tracking branches, but does not modify your working branch. 'git pull' is effectively 'git fetch' followed by a merge (or rebase, with --rebase) into your current branch. Fetch is the safer of the two since it lets you review incoming changes before integrating them.

3. Explain the difference between 'git merge' and 'git rebase'.

'git merge' combines two branches by creating a new merge commit that has two parents, preserving the full history of both branches exactly as it happened. 'git rebase' replays your branch's commits one by one on top of another branch's tip, producing a linear history without a merge commit. Merge preserves context and is non-destructive; rebase produces cleaner history but rewrites commit hashes, so it should be avoided on commits already shared with others.

4. What is a merge conflict, and how would you resolve one?

A merge conflict occurs when Git can't automatically reconcile changes because the same lines (or a file's existence) were modified differently on both branches being merged. Git marks the conflicting sections in the affected files with <<<<<<<, =======, and >>>>>>> markers. To resolve it, you edit the files to keep the correct content, remove the conflict markers, stage the resolved files with 'git add', and then complete the merge with 'git commit' (or 'git rebase --continue' during a rebase).

5. What does 'git clone' do, and how is it different from 'git init'?

'git clone ' copies an existing remote repository, including its full history and branches, into a new local directory and automatically sets up a remote called 'origin' pointing back to the source. 'git init' instead creates a brand-new, empty Git repository in the current directory with no history and no configured remote — it's used to start version-controlling a project from scratch.

Staging, Commits & History

1. What is the staging area (index) in Git, and why does it exist?

The staging area (or index) is an intermediate layer between your working directory and the repository's commit history. Running 'git add' moves changes from the working directory into the staging area, and 'git commit' then records exactly what's staged. It exists so you can build a commit deliberately — choosing precisely which changes (even specific hunks, via 'git add -p') go into the next commit rather

than being forced to commit everything you've changed at once.

2. How do you undo a commit that hasn't been pushed yet?

'git reset --soft HEAD~1' undoes the commit but keeps the changes staged. 'git reset --mixed HEAD~1' (the default) undoes the commit and unstages the changes, leaving them in the working directory. 'git reset --hard HEAD~1' undoes the commit and discards the changes entirely. Since the commit hasn't been pushed, rewriting local history this way is safe.

3. What is the difference between 'git reset', 'git revert', and 'git checkout'?

'git reset' moves the current branch pointer to a different commit, optionally altering the staging area and working directory — it rewrites history and is meant for local/unpushed commits. 'git revert' creates a new commit that undoes the changes of a previous commit, preserving history — it's safe for already-shared commits. 'git checkout' (or 'git switch'/'git restore' in newer Git) is used to switch branches or restore files to a particular state without altering commit history.

4. How would you squash multiple commits into one?

Run 'git rebase -i HEAD~N' where N is the number of commits to squash. In the interactive editor, leave the first commit as 'pick' and change the subsequent commits' action from 'pick' to 'squash' (or 's'). Git will then combine them and prompt you to write a single, unified commit message.

5. What does 'git cherry-pick' do, and when would you use it?

'git cherry-pick ' applies the changes introduced by a specific commit from one branch onto another, creating a new commit with the same changes (but a new hash). It's useful when you need just one fix or feature from a branch — for example, backporting a bugfix to a release branch — without merging the entire branch.

Branching & Collaboration

1. What is a branch in Git, and how is it implemented internally?

A branch is simply a movable, lightweight pointer to a specific commit. Internally, it's just a file containing a commit hash, stored under .git/refs/heads/. Because branches are just pointers rather than copies of the codebase, creating and switching branches in Git is fast and cheap compared to systems where branching duplicates files.

2. Explain a common branching strategy (e.g., Git Flow or trunk-based development).

Git Flow uses long-lived 'main' and 'develop' branches, with dedicated feature branches created off 'develop' and merged back in, plus release and hotfix branches for stabilizing and shipping — it suits scheduled release cycles. Trunk-based development instead has everyone working off a single main branch with short-lived feature branches (often merged within a day via small, frequent commits), relying on feature flags and CI to keep 'main' always releasable — it suits continuous delivery.

3. How do you resolve a conflict during a rebase?

When a rebase stops due to a conflict, edit the conflicting files to resolve the markers, stage the fixed files with 'git add', and then run 'git rebase --continue' to proceed to the next commit being replayed. If you want to abandon the rebase entirely, 'git rebase --abort' returns the branch to its state before the rebase started.

4. What is a fast-forward merge versus a three-way merge?

A fast-forward merge happens when the target branch's tip is a direct ancestor of the branch being merged in — Git simply moves the pointer forward, with no new merge commit created. A three-way merge happens when both branches have diverged (each has commits the other lacks); Git uses the common ancestor plus the tips of both branches to compute the merge and creates a new commit with two parents.

5. How do you delete a remote branch, and how is that different from deleting a local branch?

A local branch is deleted with 'git branch -d ' (or -D to force-delete an unmerged branch). A remote branch is deleted with 'git push origin --delete ', which tells the remote to remove its ref — this doesn't automatically remove any local copies of that remote-tracking branch, which may need pruning separately with 'git fetch --prune'.

Advanced & Internals

1. What are Git hooks, and can you give an example use case?

Git hooks are scripts that Git runs automatically at specific points in the workflow, stored in the .git/hooks directory. Common hooks include 'pre-commit' (e.g., to run linters or tests before allowing a commit) and 'pre-push' or server-side hooks like 'pre-receive' (e.g., to enforce commit message standards or block pushes that fail CI checks).

2. Explain how Git stores data internally (blobs, trees, commits).

Git's object database stores three main object types, each addressed by the SHA hash of its content. A blob stores the raw content of a file (no filename or metadata). A tree represents a directory, listing blobs and other trees along with their filenames and modes. A commit points to a single tree (representing the project's full state at that point), references its parent commit(s), and stores metadata like author, committer, and message.

3. What is 'git stash', and how would you use it in a typical workflow?

'git stash' temporarily shelves uncommitted changes (both staged and unstaged) so you can switch context — for example, to switch branches and fix an urgent bug — without committing incomplete work. The changes are saved on a stack; 'git stash pop' reapplies the most recent stash and removes it from the stack, while 'git stash apply' reapplies it but keeps it stashed.

4. What is the difference between 'git rebase -i' and a regular rebase?

A regular rebase ('git rebase ') simply replays your commits on top of another branch automatically, commit by commit, with no opportunity to alter them. Interactive rebase ('git rebase -i ') opens an editable list of the commits being replayed, letting you reorder, reword messages, squash, split, or drop individual commits before they're reapplied.

5. What is a detached HEAD state, and how do you recover from it?

A detached HEAD occurs when HEAD points directly to a specific commit rather than to a branch — typically after checking out a commit hash, a tag, or a remote branch directly. Any new commits made in this state aren't attached to a branch and can be lost once you check out something else, since nothing keeps a reference to them. To recover, create a branch at the current commit with 'git branch ' (or 'git

switch -c ') before switching away, which anchors those commits permanently.